

Two Dimensional CNN Replication Code Methodology

NSL KDD and UNSW NB15

Purpose. This report records how the complete 2D CNN reproduction was designed, implemented, verified, and explained. The project owner directed the experimental scope and execution rules. The implementation converted those rules into validated preprocessing, concurrent GPU training, paper comparison, and LIME and SHAP scripts.

User Directed Replication Rules

Rule	Reason and implementation
Follow the paper first	Keep every architecture and training value that the paper states explicitly.
Test all relevant undefined choices	Do not select one convenient value when the paper is unclear. Build a finite matrix of plausible options.
Use 64 random seeds per configuration	Measure stochastic variation and prevent one favorable or unfavorable run from deciding the result.
Launch every model concurrently on the GPU	Execute 12 x 64 NSL-KDD models and 6 x 64 UNSW-NB15 models as concurrent replicas.
Show progress	Print epoch status; the UNSW-NB15 runner also shows percentage, batch, elapsed time, and ETA.
Run datasets sequentially	Use PowerShell to finish training and comparison for one dataset before starting the next.
Compare automatically with the paper	Rank individual runs and 64-seed configuration means against the published metrics.
Reproduce the explanation stage	Apply the paper's LIME and SHAP example-class protocol to the selected reproduced 2D CNN checkpoints.
Write code only and use the existing environment	The scripts contain no package installer, CUDA installer, dataset downloader, or network operation.

Why Sixty Four Seeds

- CNN accuracy changes with weight initialization, the stratified split, and epoch shuffling.
- A single seed can appear to replicate the paper by chance or miss it despite a correct implementation.
- Using the same 64 seeds for every configuration creates a controlled comparison because corresponding runs begin from the same seed conditions.
- The outputs include the mean, standard deviation, minimum, maximum, and 95 percent confidence interval for each configuration.

Fixed Paper Settings

Setting	Value used
Convolutions	64 to 32 to 32 filters; 3 x 3 kernels; same padding; ReLU
Pooling and output	2 x 2 max pooling after each convolution; five output classes
Training	20 epochs; learning rate 0.001; weight decay 0.0001
Loss	Sparse multiclass cross entropy
Split	Stratified 60 percent train, 15 percent validation, 25 percent test
Input	NSL-KDD 6 x 6 x 1; UNSW-NB15 7 x 7 x 1

Undefined Choices and Configuration Matrix

Choice	Options	Reason
Batch size	32, 64, 128	The paper did not define one batch size.
Weight decay semantics	AdamW or Adam with L2	Both are plausible interpretations of the stated weight decay.
NSL-KDD input feature 36	Retain difficulty or append one zero	After removing the six named predictors, the paper does not explain how its 6 x 6 input retains 36 values.
UNSW-NB15 loss field	Retain sloss and dloss	The paper names loss, but the supplied files have no such column. Retaining both produces the stated 38 predictors and 11 padding values.

Dataset	Matrix calculation	Configs	Seeds	Concurrent models
NSL-KDD	3 batches x 2 feature modes x 2 optimizers	12	64 each	768
UNSW-NB15	3 batches x 2 optimizers	6	64 each	384

Data Preparation Code

Dataset	Processing performed before the CNN
NSL-KDD	Read 125,973 KDDTrain+ rows; map attacks to DoS, Normal, Probe, R2L, and U2R; remove the six paper-named predictors; integer-encode three categorical columns; min-max scale; apply the selected 36th-input interpretation; reshape to 6 x 6 x 1.
UNSW-NB15	Combine 257,673 supplied rows; keep five paper classes; cap Generic and Normal at 50,000 each to obtain 185,124 rows; remove identifiers and the supported paper-named predictors; integer-encode categorical columns; min-max scale 38 predictors; append 11 zeros; reshape to 7 x 7 x 1.

Split rule. Each training seed creates its own deterministic stratified 60/15/25 partition. UNSW-NB15 class capping uses the recorded sample seed 20260909 before those splits.

Model Code

- Three grouped 3 x 3 convolution stages reproduce the 64 to 32 to 32 filter sequence while keeping every replica's parameters separate.
- Ceiling-mode 2 x 2 pooling reproduces same-padded pooling and allows three pooling stages on the small 6 x 6 and 7 x 7 inputs.
- The final dense layer returns five logits. Cross entropy applies the softmax calculation internally, and prediction uses the largest logit.

Concurrent GPU Implementation

Mechanism	How it preserves the experiment
Grouped convolutions	One CUDA operation processes the configuration and seed dimensions, while each CNN owns independent weights. Corresponding configurations reuse the same seed initialization.
Microbatching and accumulation	A physical microbatch of 32 keeps all replicas in GPU memory. Updates after 1, 2, or 4 microbatches preserve effective batch sizes 32, 64, and 128.
Optimizer isolation	Each configuration has its own AdamW or coupled-L2 Adam optimizer state.
Mixed precision	Bfloat16 autocast reduces GPU memory and computation cost; counts and reported aggregates use higher-precision accumulation where required.
Progress and recovery	Training reports progress and writes an epoch checkpoint that is validated against the configuration and seed list before resuming.

Randomness and Reproducibility

Item	Recorded rule
Seed generation	64 unique integers sampled without replacement from a recorded master seed
Master seeds	NSL-KDD 20260908; UNSW-NB15 20260909
Per-seed effects	Initialization, stratified partition, and deterministic epoch shuffle
Audit files	Manifest, all histories, all seed results, configuration summary, and comparison outputs

Paper Comparison Rules

- Evaluate accuracy, loss, and precision, recall, and F1 for all five classes, giving 17 paper metrics; rank every individual run and every 64-seed configuration mean.
- Use rounding-aware RMSE first because the paper publishes rounded values, then raw RMSE as the tie-breaker.
- Treat accuracy as replicated when the observed value rounds to the paper's printed precision. Keep the full-metric comparison separate so an accuracy match is not reported as an exact match for every metric.

Explanation Reproduction Method

Scope. The source paper applied LIME and SHAP to its DNN. We adapted its example classes to the reproduced 2D CNN: NSL-KDD Normal and DoS for LIME and DoS for local SHAP; UNSW-NB15 Normal and Exploits for LIME and Normal for local SHAP.

Decision	Rule used
Model selection	Use the individual run with the closest 17-metric profile to the paper, not the run chosen only by accuracy.
Checkpoint extraction	Extract one replica's weights from the grouped epoch-20 checkpoint and rebuild the ordinary single-model 2D CNN.
Inference verification	Require extracted-model accuracy to be within 0.0005 of the stored grouped-model accuracy.
LIME perturbation	For each real feature, either keep the instance value or replace it with the training mean; weight nearby samples more heavily.
LIME surrogate	Fit weighted ridge regression to 5,000 perturbations and report feature weights, weighted R2, RMSE, and MAE.
Kernel SHAP local	Fit model-agnostic coalition regression with 4,096 coalitions and a training-mean baseline.
Kernel SHAP global	Average absolute predicted-class SHAP values across 20 stratified test cases with 768 coalitions per case.
Padding treatment	Keep zero-padding cells fixed in every model input and exclude them from the interpretable feature set.

Dataset	Selected run	Stored accuracy	Verified accuracy	Explained features
NSL-KDD	batch032-zero_pad-adam_l2 / 1414324351	0.992030228	0.992157236	35 of 36
UNSW-NB15	batch032-adamw / 1349501436	0.814325843	0.814174589	38 of 49

Explanation Verification

- Local SHAP reconstruction matched the model probability for both datasets; the efficiency errors were zero within floating-point precision.
- No constant padding feature appears in the LIME or SHAP explanation tables.
- Five unit tests cover checkpoint configuration reconstruction, additive Kernel SHAP recovery, LIME coefficient signs and fidelity, padding exclusion, and weighted ridge recovery.
- All eight generated explanation plots and every CSV and JSON output were checked for expected rows, dimensions, and finite values.
- LIME fidelity is reported rather than hidden. Saturated Normal predictions have low weighted R2 because their local probabilities vary little, although their weighted RMSE remains small.

Output Record

Output	Contents
Training directories	All seed results, per-epoch histories, configuration summaries, manifests, checkpoints, and paper-comparison rankings
outputs xai 2dcnn	LIME plots and CSV, local SHAP plots and CSV, global SHAP plots and CSV, instance-level SHAP data, and JSON audit summaries
Reports	Concise results report and this code methodology report

Code Responsibilities

Files	Responsibility
nsl_kdd_2dcnn.py and unsw_nb15_2dcnn.py	Dataset validation, label mapping, feature preparation, and reference CNN definition
nsl_kdd_gpu_sweep.py and unsw_nb15_gpu_sweep.py	Configuration matrix, random seed selection, vectorized CNN components, and metrics
nsl_kdd_gpu_all_concurrent.py and unsw_nb15_gpu_all_concurrent.py	All-model GPU training, accumulation, progress, checkpoints, evaluation, and CSV output
compare_results_to_paper.py and compare_unsw_results_to_paper.py	Rounding-aware and raw metric ranking against the paper
explain_2dcnn.py	Selected-checkpoint extraction, accuracy verification, LIME, local and global Kernel SHAP, plots, CSV output, and JSON audit metadata
test_explain_2dcnn.py	Unit tests for checkpoint interpretation, LIME, Kernel SHAP, padding exclusion, and weighted regression
build_replication_report.py and build_code_methodology_report.py	Deterministic generation of the two final Word reports from recorded outputs

Contribution Record

Project owner. Selected the paper and datasets; supplied the target hyperparameters; required all relevant undefined choices, 64 seeds, full GPU concurrency, progress reporting, sequential execution, automated comparison, and code-only operation with no downloads.

Implementation support. Inspected the paper and datasets; translated the rules into preprocessing and training code; added validation, GPU vectorization, checkpointing, metrics, tests, comparison scripts, explanation code, and reports.

Outcome. The user-directed reproduction matched the paper's reported 2D CNN accuracy precision on both datasets. It also produced verified LIME and Kernel SHAP explanations for the selected reproduced checkpoints, with the paper-to-2D-CNN scope adaptation stated explicitly.